Лабораторная работа №3

Модель данных в OpenMP

Пояснения к программам и ответы на вопросы

Проект ParallelProgramming | C++17 | Visual Studio 2022
Проверено: 16 октября 2026 года

Цель работы

Разобраться, какие данные доступны всем нитям, какие существуют отдельно для каждой нити, как передавать начальные значения и как получать результаты после параллельной области. Выполнены примеры 12-16 из задания; для пояснения вопросов добавлены три небольших примера.

Пункт	Функция в Lab3.cpp	Что показывает
1	showPrivate()	Отдельные копии без копирования начального значения
2	showShared()	Изменение разных элементов общего массива
3	showFirstprivate()	Локальные копии с начальным значением
4	showThreadprivate()	Сохранение копий между двумя областями
5	showCopyin()	Копирование значения мастера в threadprivate
6	showLastprivate()	Возврат значения последней логической итерации
7	showReduction()	Безопасное объединение частичных сумм
8	showCopyprivate()	Рассылка результата исполнителя single

Как запустить

Открыть ParallelProgramming.sln, выбрать x64 и Debug либо Release, собрать проект и запустить программу. В главном меню выбрать **3**, затем **9** для всех примеров или пункт 1-8 для отдельного примера. Ввод **0** возвращает назад; ещё один 0 завершает программу.

В коде сохранены простые функции, Allman-скобки и switch-меню. Ответы на теоретические вопросы не выводятся программой: они находятся только в этом документе.

1. private и shared

showPrivate(): своя переменная у каждой нити

Во внешней последовательной области создаётся value = 1. Опция private(value) создаёт для каждой нити отдельный экземпляр value. Это не ссылка на внешний объект и не его инициализированная копия. Для обычного int начальное значение приватного экземпляра не задано.

```
int value = 1;
#pragma omp parallel num_threads(4) private(value)
{
    value = omp_get_thread_num();
    // Здесь можно читать value: присваивание уже выполнено.
}
```

Внутри области каждая нить сначала записывает в свою value собственный номер, а затем печатает значение. Получаются значения 0, 1, 2 и 3. Изменения не затрагивают внешний объект, поэтому после завершения области он по-прежнему равен 1.

Исправление примера 12: в методичке предлагается напечатать private-переменную до присваивания. Для int в используемом C++17 это чтение неинициализированного значения и неопределённое поведение, а не корректный эксперимент со «случайным числом». В рабочей программе такое чтение исключено.

showShared(): один общий массив

Массив values из десяти int заполняется нулями. В команде запрашиваются четыре нити, и каждая выполняет values[thread] = 1. В отличие от private, массив не размножается: все нити обращаются к одному объекту, но используют разные элементы.

```
int values[10] = {};
#pragma omp parallel num_threads(4) shared(values)
{
    values[omp_get_thread_num()] = 1;
}
```

При четырёх нитях результат равен **1 1 1 1 0 0 0 0 0 0**. Здесь нет гонки: разные элементы массива int являются разными ячейками памяти. После выхода из parallel все записи завершены, и последовательная часть может читать итоговый массив.

Ограничение команды необходимо: без num_threads(4) пример методички на устройстве с числом нитей больше 10 мог бы выйти за границы массива. Код также измеряет фактический размер команды через single и проверяет число заполненных элементов. critical защищает строки вывода в других примерах, но сам по себе shared синхронизацию не предоставляет.

2. firstprivate и copyin

showFirstprivate(): локальное значение на входе

Перед областью value равна 1. Опция firstprivate(value) не просто создаёт отдельные переменные, но и инициализирует каждую значением внешней value. Поэтому все нити начинают с 1. Затем каждая записывает свой номер: 0, 1, 2 или 3.

```
int value = 1;
#pragma omp parallel num_threads(4) firstprivate(value)
{
    int before = value;
    value = omp_get_thread_num();
}
```

В программе before сохраняет входное значение для вывода и проверки. Счётчик errors с reduction(+:errors) позволяет безопасно подсчитать нарушения ожидаемого результата. Внешняя value не меняется и после области остаётся равной 1: firstprivate копирует данные только на входе, а не обратно на выходе.

showCopyin(): начальное значение постоянных копий

Переменная threadValue объявлена со статической длительностью хранения и отмечена threadprivate. В последовательной области присваивание threadValue = 1 меняет только экземпляр начальной нити. copyin(threadValue) при входе в parallel копирует это значение в экземпляры остальных участников команды.

```
threadValue = 1;
#pragma omp parallel num_threads(4) copyin(threadValue)
{
    int before = threadValue;
    threadValue += omp_get_thread_num();
}
```

Все входные значения равны 1. После прибавления номера нити получаются 1, 2, 3 и 4. Это независимые экземпляры, поэтому изменение одной нитью не меняет остальные. После области последовательная часть видит экземпляр начальной нити: 1 + 0 = 1.

Свойство	firstprivate	copyin
Какие данные	Обычная переменная, допустимая для этой опции	Только threadprivate-переменная
Источник	Внешний объект в контексте входящей нити	Экземпляр мастера новой команды
Результат	Новые приватные экземпляры конструкции	Обновлённые экземпляры threadprivate
После области	Автоматического возврата нет	Жизнь копий связана с нитями; сохранение условно

3. threadprivate: память между областями

showThreadprivate(): последовательность действий

Обычный private привязан к выполнению конструкции. threadprivate задаёт отдельный экземпляр переменной для каждой нити и позволяет использовать его в нескольких областях. threadValue определена в анонимном пространстве имён, имеет статическую длительность хранения и недоступна по имени из других исходных файлов.

```
namespace
{
    int threadValue = 0;
#pragma omp threadprivate(threadValue)
}
```

Сначала подготовительная команда обнуляет свои экземпляры. Это делает демонстрацию повторяемой даже после copyin или предыдущего запуска пункта 4: нельзя предполагать, что повторно используемые рабочие нити снова имеют исходные нули. Затем начальная нить устанавливает только свой экземпляр в 1.

В первой демонстрационной области каждая нить сохраняет входное значение, присваивает threadValue свой номер и выводит оба значения. Между областями программа печатает экземпляр начальной нити. Во второй области значения только читаются и сравниваются с номерами нитей.

Нить	Первая область: вход	Первая: выход	Вторая: вход
0	1	0	0
1	0	1	1
2	0	2	2
3	0	3	3

Между областями threadValue равна **0**, а не сумме 0 + 1 + 2 + 3: последовательная часть обращается только к своему экземпляру. Чтобы собрать все результаты в одном месте, нужен общий массив или другая явная передача данных.

Когда значения разрешено использовать повторно

Для схемы из этой работы области не вложены в другую активную parallel-область, запрашивают одинаковую команду и выполняются при отключённом динамическом изменении её размера. В runLab3() вызывается omp_set_dynamic(0), а при выходе восстанавливается прежняя настройка. Код отдельно сравнивает фактические размеры двух команд.

Это не безусловная гарантия для любых программ: если менять команду или нарушать условия сохранения threadprivate, значения рабочих нитей нельзя считать надёжно перенесёнными. В современных версиях OpenMP учитываются также одинаковая политика привязки нитей и отсутствие операций, для которых стандарт снимает такую гарантию. В этом проекте используется обычный режим MSVC /openmp и простая последовательность областей.

4. Возврат и передача результатов

showLastprivate(): последнее по порядку, а не по времени

Цикл перебирает i от 0 до 11 и вычисляет value = i * i. Опция lastprivate(value) создаёт приватные экземпляры и после цикла переносит во внешний объект значение из последней логической итерации. В последовательном порядке это i = 11, поэтому итог всегда равен **121**, если выполнение корректно.

```
#pragma omp parallel for num_threads(4) \
    schedule(static, 1) lastprivate(value)
for (int i = 0; i < 12; i++)
{
    value = i * i;
}
```

При четырёх нитях schedule(static, 1) раздаёт итерации по кругу: нить 0 получает 0, 4, 8; нить 1 - 1, 5, 9; нить 2 - 2, 6, 10; нить 3 - 3, 7, 11. Какая нить физически закончит последней, неважно. Исходное значение -1 не копируется в приватные экземпляры; в этом примере каждое чтение выполняется после присваивания.

showReduction(): объединение частичных результатов

Для суммы чисел от 1 до 1 000 000 нельзя просто выполнять общий sum += i без защиты: это чтение, сложение и запись, которые могут конфликтовать. reduction(+:sum) создаёт локальные аккумуляторы с нейтральным значением 0 и затем объединяет их сложением с исходной sum, которая здесь тоже равна 0.

```
long long sum = 0;
#pragma omp parallel for num_threads(4) reduction(+:sum)
for (int i = 1; i <= 1000000; i++)
{
    sum += i;
}
```

Контрольная формула N * (N + 1) / 2 даёт **500000500000**. Использован long long, поскольку результат не помещается в 32-битный int. Приведение к long long выполняется до умножения в контрольной формуле, чтобы не переполнить промежуточное значение.

showCopyprivate(): от исполнителя single ко всем

Внутри parallel каждая нить создаёт свои value = -1 и source = -1. Одна нить в single copyprivate(value, source) устанавливает value = 42 и source в собственный номер. После single эти значения скопированы всем участникам. Исполнитель может быть любым, не обязательно мастером. Все строки должны содержать 42 и один и тот же source.

У single с copyprivate есть синхронизация завершения рассылки; добавлять nowait к этой конструкции нельзя. Это передача между уже работающими нитями, а не сохранение данных после завершения команды.

5. Ответы на вопросы 1 и 2

1. Может ли одна и та же переменная выступать в одной части программы как общая, а в другой части - как локальная?

Да. Атрибут совместного доступа определяется контекстом и конкретной конструкцией OpenMP. Например, обычную внешнюю автоматическую переменную можно использовать как shared в одной parallel-области и как private или firstprivate в другой.

```
int value = 10;
#pragma omp parallel num_threads(4) shared(value)
{
    // Все нити читают один внешний объект value.
}
#pragma omp parallel num_threads(4) firstprivate(value)
{
    value += omp_get_thread_num();
}
```

Важно различать имя и объект. В первой области имя value обозначает одну общую переменную. Во второй то же имя обозначает разные приватные экземпляры. Внешний объект при этом продолжает существовать, но обращения внутри области направлены к копиям. После второй области внешняя value остаётся равной 10.

Смена атрибутов не произвольна: действуют ограничения OpenMP. Например, переменную с threadprivate нельзя просто объявить shared или firstprivate в списке такой конструкции. Для рассылки её значения предусмотрен copyin.

2. Что произойдёт, если несколько нитей одновременно обратятся к общей переменной?

Ответ зависит от вида обращений и синхронизации. Если все только читают корректно инициализированное значение и параллельной записи нет, само совместное чтение допустимо. Если хотя бы одна нить пишет в ту же ячейку, а конфликтующие обращения не упорядочены нужной синхронизацией, возникает гонка данных. Нельзя обещать ни конкретное значение, ни даже «один из нескольких ожидаемых результатов».

Пример: counter++ обычно состоит из чтения, увеличения и записи. Две нити могут прочитать одинаковый старый counter и обе записать старое значение плюс один. Один инкремент окажется потерян. Даже одинаковые значения, записываемые в одну ячейку разными нитями, сами по себе не делают обращение безопасным.

Для одного поддерживаемого обновления можно использовать atomic; для нескольких связанных действий - critical или блокировку; для накопления суммы - reduction. Барьер подходит для разделения фаз записи и чтения, но не исправляет гонку, уже возникшую внутри фазы. В showShared() нити пишут в разные элементы int-массива, поэтому именно такой гонки нет.

6. Ответы на вопросы 3 и 4

3. Может ли произойти конфликт, если несколько нитей одновременно обратятся к одной и той же локальной переменной?

Если каждая нить обращается к своему приватному экземпляру, конфликта между этими экземплярами нет. Одинаковое имя переменной в исходном коде не означает одну ячейку памяти. Например, в showPrivate() записи value = 0 и value = 3 выполняются в разных объектах и друг другу не мешают.

Однако слово «локальная» не заменяет анализ адресов. Если адрес одного объекта передать другим нитям и они начнут обращаться именно к нему, конфликт может появиться. Аналогично, приватные указатели могут хранить один и тот же адрес общего объекта.

```
int common = 0;
int* pointer = &common;
#pragma omp parallel num_threads(4) firstprivate(pointer)
{
    // pointer свой, но *pointer обозначает общий common.
    // Запись в *pointer потребовала бы синхронизации.
}
```

firstprivate(pointer) копирует значение адреса, но не создаёт отдельную копию объекта по этому адресу. Для настоящей независимости нужны разные целевые объекты или непересекающиеся участки памяти. Это относится и к динамически выделенным массивам.

4. Каким образом при входе в параллельную область разослать всем порождаемым нитям значение некоторой переменной?

Для обычной переменной используется **firstprivate**. Внешнее значение копируется в приватные экземпляры участников команды до выполнения их кода. В showFirstprivate() все участники получают 1, а затем независимо меняют свои значения.

Если переменная заранее объявлена **threadprivate**, используется **copyin**. Он копирует экземпляр мастера команды в экземпляры остальных нитей на входе в parallel. Пример - showCopyin(): все входные значения threadValue равны 1.

Если нужна не отдельная копия, а только доступ к неизменяемому значению, иногда достаточно shared и отсутствия записей. Это общее чтение, а не рассылка независимых копий. Если значение вычисляется уже внутри команды одной нитью, подходит single copyprivate, как в пункте 8, а не copyin на входе.

Выбор механизма определяется не только одинаковым начальным числом, но и требуемым временем жизни: новая копия на одну конструкцию, постоянный экземпляр нити или передача результата между участниками текущей команды.

7. Ответ на вопрос 5

5. Можно ли сохранить значения локальных копий общих переменных после завершения параллельной области? Если да, то что необходимо для их использования?

Да, но нужно явно выбрать, что именно сохранить. Обычный private и обычный firstprivate сами по себе не возвращают значения во внешний объект и не предоставляют способ затем получить прежние экземпляры. После окончания конструкции нельзя рассчитывать на их доступность или содержимое в следующей области.

Вариант 1. Сохранить отдельное состояние каждой нити

Использовать threadprivate для переменной со статической длительностью хранения. Каждая нить обращается к собственному экземпляру и может использовать его в следующей области при выполнении условий сохранения. В схеме методички области не должны быть вложенными в другую активную parallel-область, размеры команд должны совпадать, а динамическое изменение размера команды должно быть выключено на обеих областях.

showThreadprivate() задаёт num_threads(4), временно отключает dynamic, проверяет фактические размеры и демонстрирует повторное чтение 0, 1, 2 и 3. Начальная нить между областями видит только собственный экземпляр. Для современных реализаций нужно учитывать полный набор условий стандарта, в том числе политику привязки нитей; одинаковый запрошенный размер сам по себе недостаточен.

Вариант 2. Получить одно итоговое значение

lastprivate подходит, если нужен результат последней логической итерации. В showLastprivate() во внешнюю value возвращается 121. Это не результат случайно последней завершившейся нити и не коллекция всех приватных значений.

reduction подходит для объединения: суммы, произведения или другой поддерживаемой операции. В showReduction() наружу выходит общая сумма 500000500000. Частичные суммы отдельных нитей после объединения программе не выдаются.

Вариант 3. Сохранить все значения явно

Выделить общий массив достаточного размера и записать результат каждой нити в отдельный элемент до выхода из области. Последовательная часть затем прочитает массив после завершения команды. Индекс и размер должны быть проверены; если несколько нитей могут писать в один элемент, потребуется синхронизация.

Этот способ сохраняет обычные данные в объекте с подходящим временем жизни, а не идентичность конкретной рабочей нити. Для переноса результатов между произвольно меняющимися командами он обычно понятнее, чем зависимость от threadprivate.

Главное различие: threadprivate сохраняет состояние экземпляров нитей при условиях; lastprivate выбирает один результат; reduction объединяет результаты; общий массив позволяет хранить их все. Это разные задачи, а не взаимозаменяемые опции.

8. Ответ на вопрос 6

6. В чём отличие опции copyin от опции firstprivate?

Обе опции дают нитям одинаковое начальное значение, но работают с разными видами переменных и по-разному связаны с временем жизни их экземпляров.

Критерий	firstprivate	copyin
К чему применяется	К допустимой обычной переменной в конструкции OpenMP	К переменной, заранее объявленной threadprivate
Что создаёт	Приватные экземпляры на время выполнения конструкции	Не заменяет threadprivate; задаёт начальные значения его экземплярам
Откуда берётся значение	Из внешнего объекта в контексте входящей нити	Из экземпляра мастера команды на входе в parallel
Независимость изменений	Изменения одной копии не меняют остальные	То же: экземпляры threadprivate независимы
После окончания области	Нет автоматического копирования обратно	Начальная нить видит свой экземпляр; рабочие сохраняют свои при условиях
Для чего удобно	Передать параметры одному параллельному вычислению	Инициализировать постоянное состояние каждой нити

Как это видно в реализованных примерах

showFirstprivate() создаёт обычную value = 1. Все нити получают 1, присваивают свои номера и завершают работу. Внешняя value остаётся 1, даже хотя нить 0 внутри области присвоила своей копии 0. Её приватная value не является внешним объектом.

showCopyin() работает с threadValue. Начальная нить устанавливает свой экземпляр в 1, copyin передаёт 1 другим участникам, затем все прибавляют свой номер. После области начальная нить видит собственную threadValue = 1. Если бы она изменила её ещё раз внутри области, последовательная часть увидела бы именно это новое значение её экземпляра.

copyin не собирает данные назад. Он выполняет входное копирование. Постоянство экземпляров обеспечивает threadprivate, причём не безусловно. И firstprivate, и copyin не являются заменой reduction или lastprivate.

Копирование указателя в обоих случаях не означает глубокого копирования памяти по адресу. Если экземпляры указателя содержат один адрес, нити всё ещё могут обращаться к одному общему объекту и нуждаться в синхронизации.

9. Проверка и перенос проекта

Что проверено фактическим запуском

Проект успешно собран MSBuild из Visual Studio 2022 в конфигурациях **Release x64** и **Debug x64**. Выполнены 36 сценариев запуска процесса; сообщения об ошибке проверки не получены. Это проверка корректности примеров, а не замер ускорения.

Проверка	Полученный результат
Все примеры и отдельные пункты 1-8	Восемь успешных проверок в режиме «все»; по одной в отдельных пунктах
private / firstprivate	Внешняя value остаётся 1; у firstprivate все входные значения равны 1
shared	Четыре единицы и шесть нулей; фактическая команда из 4 нитей
threadprivate	Вторая область читает 0, 1, 2, 3; команды имеют размер 4 и 4
copyin / copyprivate	Входные значения copyin равны 1; copyprivate передаёт всем 42 и один source
lastprivate / reduction	121 и 500000500000 соответственно
Повторный запуск	После copyin и повторного входа threadprivate снова даёт ожидаемый результат
Ошибочный ввод и EOF	Меню восстанавливается после текста; неверный номер обрабатывается; EOF завершает ввод
Настройки среды и другие лабораторные	Проверены OMP_NUM_THREADS=1/12, OMP_DYNAMIC=TRUE/FALSE и переход после вложенных областей лабораторной №2

Полный вывод одного запуска Release сохранён в output/lab3_run.txt. Порядок строк от разных нитей может меняться: critical запрещает смешивать защищённые выводы, но не задаёт порядок 0, 1, 2, 3. Исполнитель single также может меняться.

Что нужно на другом устройстве

OpenMP уже включён в ParallelProgramming.vcxproj общим параметром OpenMPSupport=true для всех конфигураций. После клонирования повторно включать его вручную не требуется. Нужны Visual Studio 2022 с компонентами разработки классических приложений на C++, инструментами v143 и подходящим Windows SDK. Настройка /utf-8 и стандарт C++17 также сохранены в проекте.

Это Windows-проект: главное меню использует Windows.h. Для Linux или macOS потребуются адаптация меню и своя сборка с поддержкой OpenMP. Перенос только готового exe - отдельная задача: Release может потребовать Microsoft Visual C++ Redistributable соответствующей архитектуры; Debug не предназначен для распространения на обычные компьютеры.

Основа: предоставленная методичка «ППЛабораторная_работа_3.pdf», примеры 12-16 и вопросы 1-6. Дополнительные демонстрации поясняют механизмы, упомянутые в её теоретической части.